Assignment 4

Vector Operations Program: Implement 2D/3D vector addition, subtraction, and dot product with exception handling.
https://github.com/krrish-talesara/Programming-in-Java-Lab-

Main

```
public class VectorOperations {
    public static void main(String[] args) {
        try {
            double[] v1 = {3, 5};
            double[] v2 = {7, 2};
            double[] v3 = {3, 2, 1};
            double[] v4 = {2, 3, 9};

            Vector vector1 = new Vector(v1);
            Vector vector2 = new Vector(v2);
            Vector vector3 = new Vector(v3);
            Vector vector4 = new Vector(v4);

            Vector sum_2D = vector1.add(vector2);
            Vector diff_2D = vector1.subtract(vector2);
            double dot_2D = vector1.dot(vector2);

            Vector sum_3D = vector3.add(vector4);
            Vector diff_3D = vector3.subtract(vector4);
            double dot_3D = vector3.dot(vector4);

            System.out.println("v1: " + vector1);
            System.out.println("v2: " + vector2);
            System.out.println("v3: " + vector3);
            System.out.println("v4: " + vector4);
            System.out.println("Addition (2D): " + sum_2D);
            System.out.println("Subtraction (2D): " + diff_2D);
            System.out.println("Dot Product (2D): " + dot_2D);
            System.out.println("Addition (3D): " + sum_3D);
            System.out.println("Subtraction (3D): " + diff_3D);
            System.out.println("Dot Product (3D): " + dot_3D);
        } catch (VectorException e) {
            System.out.println("Error: " + e.getMessage());
```

```
        } catch (Exception e) {
            System.out.println("Invalid input.");
        }

    }
}
```

 # Vector

```
public class Vector {
    private double[] components;

    public Vector(double[] components) {
        if (components == null || (components.length != 2 && components.length != 3)) {
            throw new IllegalArgumentException("Vector must have 2 or 3 components.");
        }
        this.components = components.length == 2 ? new double[2] : new double[3];
        System.arraycopy(components, 0, this.components, 0, this.components.length);
    }

    private void checkDimension(Vector other) throws VectorException {
        if (this.components.length != other.components.length) {
            throw new VectorException("Vectors must have the same dimension.");
        }
    }

    public Vector add(Vector other) throws VectorException {
        checkDimension(other);
        double[] result = new double[this.components.length];
        for (int i = 0; i < this.components.length; i++) {
            result[i] = this.components[i] + other.components[i];
        }
        return new Vector(result);
    }

    public Vector subtract(Vector other) throws VectorException {
        checkDimension(other);
        double[] result = new double[this.components.length];
        for (int i = 0; i < this.components.length; i++) {
            result[i] = this.components[i] - other.components[i];
        }
        return new Vector(result);
    }
    public double dot(Vector other) throws VectorException {
        checkDimension(other);
```

```
        double result = 0;
        for (int i = 0; i < this.components.length; i++) {
            result += this.components[i] * other.components[i];
        }
        return result;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("(");
        for (int i = 0; i < components.length; i++) {
            sb.append(components[i]);
            if (i < components.length - 1) {
                sb.append(", ");
            }
        }
        sb.append(")");
        return sb.toString();
    }
}
```

VectorException

```
public class VectorException extends Exception {
    public VectorException(String message) {
        super(message);
    }
}
```

validateSameDimension

```
public class validateSameDimension extends Exception {
    public validateSameDimension(String message) {
        super(message);
    }
}
```

Output)

```
v1: (3.0, 5.0)
v2: (7.0, 2.0)
v3: (3.0, 2.0, 1.0)
v4: (2.0, 3.0, 9.0)
Addition (2D): (10.0, 7.0)
Subtraction (2D): (-4.0, 3.0)
Dot Product (2D): 31.0
Addition (3D): (5.0, 5.0, 10.0)
Subtraction (3D): (1.0, -1.0, -8.0)
Dot Product (3D): 21.0
```

Exception Handling:-

1) If 2 vectors are not of same Dimensions

```
Error: Vectors must have the same dimension for Operation to be performed.
```

2) If Vector Does not contain 2 or 3 elements

```
Error: Vector must have 2 or 3 components.
```